

# Oportunidades de refactorización en

`org.openmarkov.gui`

Catálogo, análisis y plan de ejecución

OpenMarkov — CISIAD, UNED

Abril 2026

## Índice

|                                                             |           |
|-------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                      | <b>3</b>  |
| 1.1. Estructura del módulo . . . . .                        | 3         |
| <b>2. Estado de las refactorizaciones</b>                   | <b>4</b>  |
| <b>3. R1 — Descomposición de MainPanelListenerAssistant</b> | <b>4</b>  |
| 3.1. Situación actual . . . . .                             | 4         |
| 3.2. Problemas . . . . .                                    | 6         |
| 3.3. Estado: Completada . . . . .                           | 7         |
| 3.4. Ficheros afectados . . . . .                           | 7         |
| <b>4. R2 — Consolidación de modos de creación de nodos</b>  | <b>7</b>  |
| 4.1. Estado: Completada . . . . .                           | 7         |
| <b>5. R3 — Eliminación de casts a TablePotential</b>        | <b>8</b>  |
| 5.1. Situación actual . . . . .                             | 8         |
| 5.2. Problemas . . . . .                                    | 8         |
| 5.3. Solución propuesta . . . . .                           | 9         |
| 5.4. Ficheros afectados . . . . .                           | 9         |
| <b>6. R4 — Simplificación de MainMenu</b>                   | <b>9</b>  |
| 6.1. Situación actual . . . . .                             | 9         |
| 6.2. Problemas . . . . .                                    | 10        |
| 6.3. Estado: Completada . . . . .                           | 10        |
| 6.4. Ficheros afectados . . . . .                           | 10        |
| <b>7. R5 — Consolidación de ediciones de valor en CPT</b>   | <b>10</b> |
| 7.1. Situación actual . . . . .                             | 10        |
| 7.2. Problemas . . . . .                                    | 11        |
| 7.3. Solución aplicada . . . . .                            | 11        |
| 7.4. Ficheros modificados . . . . .                         | 12        |

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>8. R6 — Consolidación de ValuesTable y variantes</b> | <b>12</b> |
| 8.1. Situación actual . . . . .                         | 12        |
| 8.2. Problemas . . . . .                                | 13        |
| 8.3. Estado: Completada . . . . .                       | 13        |
| <b>9. R7 — Simplificación de VisualNetwork</b>          | <b>13</b> |
| 9.1. Situación actual . . . . .                         | 13        |
| 9.2. Problemas . . . . .                                | 14        |
| 9.3. Estado: Completada . . . . .                       | 14        |
| <b>10.R8 — Consolidación de editores temporales</b>     | <b>15</b> |
| 10.1. Situación actual . . . . .                        | 15        |
| 10.2. Problemas . . . . .                               | 15        |
| 10.3. Decisión: descartada . . . . .                    | 15        |
| <b>11.Orden de ejecución recomendado</b>                | <b>15</b> |
| <b>12.Resumen cuantitativo</b>                          | <b>17</b> |

# 1. Introducción

El módulo `org.openmarkov.gui` contiene la interfaz gráfica de escritorio de OpenMarkov: ~320 clases y ~59 000 líneas de código Java (Swing + FlatLaf). Durante los trabajos de refactorización del núcleo (`org.openmarkov.core`) se identificaron múltiples oportunidades de mejora en la GUI que se documentan aquí.

## Propósito de este documento:

- Catalogar todas las oportunidades de refactorización detectadas en la GUI, con suficiente detalle para ejecutarlas de forma autónoma.
- Servir de guía de trabajo: cada sección es una unidad de refactorización independiente con estado, ficheros afectados, problemas concretos y solución propuesta.
- Registrar el progreso de ejecución en la tabla de estado (sección 2).

## 1.1. Estructura del módulo

La tabla 1 resume los paquetes principales, su número de clases y líneas de código.

| Paquete                           | Responsabilidad                   | Clases | Líneas |
|-----------------------------------|-----------------------------------|--------|--------|
| <code>window/</code>              | Paneles principales, controlador  | 5      | 3 241  |
| <code>window/edition/mode/</code> | Modos de edición (State pattern)  | 10     | 527    |
| <code>graphic/</code>             | View-model: VisualNode/Link/Net   | 16     | 4 854  |
| <code>dialog/common/</code>       | Diálogos base + PotentialPanel    | 38     | 8 706  |
| <code>dialog/node/</code>         | Propiedades de nodo               | 22     | 6 588  |
| <code>dialog/network/</code>      | Propiedades de red                | 14     | 2 412  |
| <code>dialog/treeadd/</code>      | Editor TreeADD                    | 20     | 3 251  |
| <code>menutoolbar/</code>         | Menús y barras de herramientas    | 22     | 6 252  |
| <code>action/</code>              | PNEdit GUI-específicos (Command)  | 18     | 2 312  |
| <code>component/</code>           | Tablas de CPT (ValuesTable, etc.) | 20     | 4 197  |
| <code>loader/</code>              | Iconos y cursores                 | 14     | 879    |
| <code>localize/</code>            | Internacionalización              | 5      | 277    |
| <code>validator/</code>           | Validadores de edición            | 6      | 337    |

Tabla 1: Paquetes principales de `org.openmarkov.gui`.

La figura 1 muestra las dependencias entre los subsistemas principales.

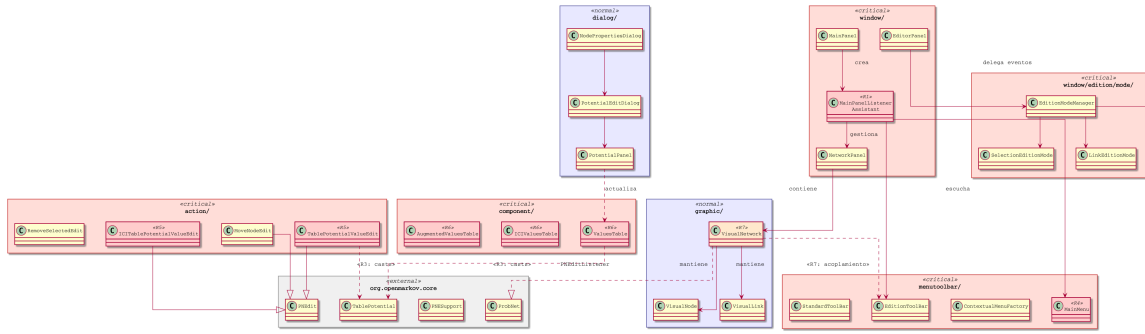


Figura 1: Dependencias entre subsistemas de la GUI. Los paquetes en rojo concentran las oportunidades de refactorización más críticas. Diagrama generado con PlantUML desde overview.puml.

## 2. Estado de las refactorizaciones

| #  | Refactorización                              | Prioridad | Esfuerzo | Estado     |
|----|----------------------------------------------|-----------|----------|------------|
| R1 | Descomposición de MainPanelListenerAssistant | Alta      | Alto     | Completada |
| R2 | Consolidación de modos de creación de nodos  | Alta      | Bajo     | Completada |
| R3 | Eliminación de casts a TablePotential        | Media     | Medio    | Completada |
| R4 | Simplificación de MainMenu                   | Media     | Medio    | Completada |
| R5 | Consolidación de ediciones de valor en CPT   | Media     | Medio    | Completada |
| R6 | Consolidación de ValuesTable y variantes     | Baja      | Alto     | Completada |
| R7 | Simplificación de VisualNetwork              | Baja      | Medio    | Completada |
| R8 | Consolidación de editores temporales         | Baja      | Bajo     | Descartada |

Tabla 2: Estado de las refactorizaciones. Actualizar la columna *Estado* a medida que se ejecutan: Pendiente → En curso → Completada.

## 3. R1 — Descomposición de MainPanelListenerAssistant

### 3.1. Situación actual

MainPanelListenerAssistant (1534 líneas) es el controlador central de la GUI. Implementa ActionListener y gestiona *todos* los eventos de la aplicación a través de un único método actionPerformed() con **63 ramas case** en un switch sobre ActionCommands.

```

1  @Override public void actionPerformed(ActionEvent e) {
2      String actionCommand = e.getActionCommand();
3      ActionCommands constant = ActionCommands.of(actionCommand);
4      switch (constant) {
5          case NEW_NETWORK      -> createNewNetwork();
6          case OPEN_NETWORK    -> executeUIAction(this::
              openNetwork);
7          case SAVE_NETWORK    -> executeUIAction(() ->
              saveNetwork(...));

```

```

8         case CLIPBOARD_COPY    -> { ... }
9         case CLIPBOARD_CUT     -> { ... }
10        case UNDO               -> undo();
11        case REDO               -> redo();
12        case CHANGE_WORKING_MODE -> { ... }
13        // ... 50+ ramas más
14    }
15 }

```

Listado 1: Fragmento de `actionPerformed()` — 67 ramas case en un único switch

Adicionalmente, la clase contiene **12 bloques try-catch** repartidos en métodos privados. Muchos siguen el patrón mecánico:

```

1 private void undo() {
2     try {
3         undoRedo(true);
4     } catch (CannotUndoException e) {
5         throw new UnrecoverableException(e);
6     }
7 }
8 private void redo() {
9     try {
10        undoRedo(false);
11    } catch (CannotRedoException e) {
12        throw new UnrecoverableException(e);
13    }
14 }

```

Listado 2: Patrón repetitivo de try-catch con `UnrecoverableException`

El bloque más complejo es el del cambio de modo de trabajo (edición ↔ inferencia), con un try-catch anidado de 6 excepciones.

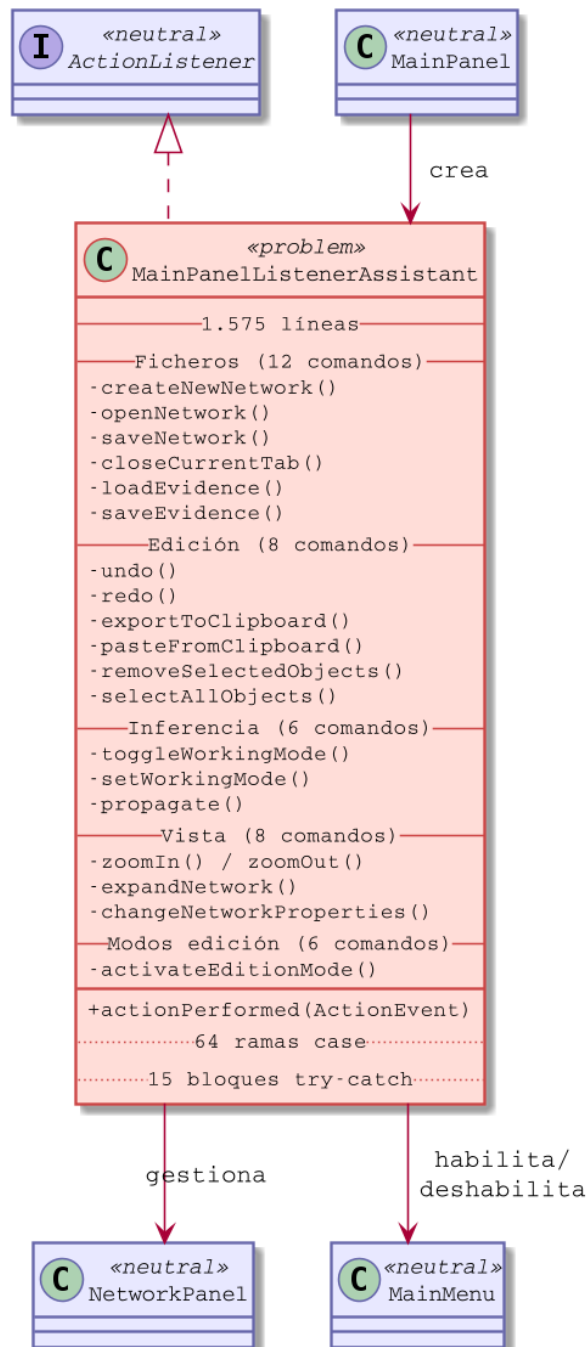


Figura 2: Estructura actual de `MainPanelListenerAssistant`: clase monolítica que gestiona ficheros, edición, inferencia y vista. Diagrama generado desde `r1-listener-actual.puml`.

### 3.2. Problemas

1. **Clase God Object**: 1 534 líneas con responsabilidades mezcladas (I/O de ficheros, edición, inferencia, vista, undo/redo).
2. **Switch monolítico**: 63 ramas case hacen difícil localizar y modificar un comportamiento concreto.

3. **Try-catch mecánicos**: 12 bloques try (23 cláusulas catch) cuyo efecto principal es re-envolver excepciones en `UnrecoverableException`.
4. **Acoplamiento**: la clase conoce detalles de diálogos, formatos de fichero, algoritmos de inferencia y estado del portapapeles.

### 3.3. Estado: Completada

Descompuesto en fachada ligera + 3 clases delegadas package-private:

| Clase                                   | Responsabilidad                         | Líneas       |
|-----------------------------------------|-----------------------------------------|--------------|
| <code>MainPanelListenerAssistant</code> | Fachada: dispatch + API pública         | 401          |
| <code>NetworkFileHandler</code>         | I/O de ficheros, abrir/guardar/cerrar   | 467          |
| <code>InferenceHandler</code>           | Modo inferencia, evidencia, propagación | 238          |
| <code>EditAndViewHandler</code>         | Undo/redo, zoom, diálogos               | 121          |
| <b>Total</b>                            |                                         | <b>1 227</b> |

Tabla 3: Resultado de la descomposición de `MainPanelListenerAssistant`. Original: 1 534 líneas; reducción neta: -307 (-20 %).

El beneficio principal es la separación de responsabilidades, no la reducción de líneas. Se eliminó ~100 líneas de código comentado muerto.

### 3.4. Ficheros afectados

- `window/MainPanelListenerAssistant.java` — fachada (reescritura)
- `window/NetworkFileHandler.java` — **nuevo**
- `window/InferenceHandler.java` — **nuevo**
- `window/EditAndViewHandler.java` — **nuevo**

## 4. R2 — Consolidación de modos de creación de nodos

### 4.1. Estado: Completada

Esta refactorización se ejecutó siguiendo la Opción A: las tres subclases (`ChanceNodeEditionMode`, `DecisionNodeEditionMode`, `UtilityNodeEditionMode`) fueron eliminadas. `NodeEditionMode` es ahora una clase concreta directamente instanciable que recibe el `NodeType` como parámetro del constructor. `EditionModeManager` registra los tres modos de nodo programáticamente mediante descriptores (`NODE_MODE_DESCRIPTOR`), generando anotaciones `@EditionState` sintéticas para compatibilidad con la toolbar.

## 5. R3 — Eliminación de casts a TablePotential

### 5.1. Situación actual

El módulo GUI contiene **16 casts explícitos** a (TablePotential) y **10 comprobaciones instanceof TablePotential**, distribuidos en 9 ficheros:

| Fichero                                       | Casts     | instanceof |
|-----------------------------------------------|-----------|------------|
| action/LinkRestrictionPotentialValueEdit.java | 5         | 0          |
| dialog/link/LinkRestrictionPanel.java         | 4         | 0          |
| action/TablePotentialValueEdit.java           | 3         | 0          |
| dialog/common/TablePotentialPanel.java        | 1         | 2          |
| component/ValuesTable.java                    | 1         | 3          |
| dialog/node/PotentialEditDialog.java          | 0         | 3          |
| component/LinkRestrictionValuesTable.java     | 1         | 1          |
| component/PotentialsTablePanelOperations.java | 1         | 0          |
| dialog/common/PolicyTypePanel.java            | 0         | 1          |
| <b>Total</b>                                  | <b>16</b> | <b>10</b>  |

Tabla 4: Distribución de casts e instanceof a TablePotential en la GUI. Un total de 26 accesos inseguros.

Ejemplo representativo del patrón problemático:

```
1 // Línea 79
2 this.tablePotential = (TablePotential) link.
  getRestrictionsPotential();
3 this.lastTable = ((TablePotential) link.getRestrictionsPotential
  ())
4                  .getValues().clone();
5 // Línea 92 (en doEdit)
6 newTable = ((TablePotential) link.getRestrictionsPotential())
7            .getValues().clone();
8 // Línea 100 (en redo)
9 this.tablePotential = (TablePotential) link.
  getRestrictionsPotential();
```

Listado 3: Casts encadenados en LinkRestrictionPotentialValueEdit

### 5.2. Problemas

1. **Seguridad de tipos**: si getRestrictionsPotential() devuelve otro tipo de Potential, se produce un ClassCastException en tiempo de ejecución.
2. **Duplicación**: el mismo cast se repite 2–5 veces dentro de una misma clase.
3. **Fragilidad**: cualquier cambio en la jerarquía de Potential exige revisar manualmente los 27 puntos.



## 5.3. Solución propuesta

Dos líneas de acción complementarias:

1. **Métodos tipados en la API:** donde el contrato garantiza que el `Potential` es un `TablePotential` (por ejemplo, restricciones de enlace), añadir un método que devuelva directamente `TablePotential` en `Link`, evitando el cast en el lado de la GUI:

```
1 public TablePotential getRestrictionsTablePotential() {  
2     return (TablePotential) getRestrictionsPotential();  
3 }
```

Listado 4: Método tipado propuesto en `Link`

2. **Almacenar referencia tipada:** en clases como `LinkRestrictionPotentialValueEdit`, hacer el cast una sola vez en el constructor y almacenarlo en un campo `TablePotential`, eliminando los casts redundantes.

## 5.4. Ficheros afectados

Los 9 ficheros de la tabla 4, más:

- `org.openmarkov.core: model/network/Link.java` — añadir métodos tipados si procede

## 6. R4 — Simplificación de `MainMenu`

### 6.1. Situación actual

`MainMenu` (1 529 líneas) declara **53 campos** de tipo `JMenu/JMenuItem/JCheckBoxMenuItem` y los construye imperativamente con **43 métodos getter** con inicialización lazy que siguen un patrón repetitivo:

```
1 private JMenuItem getFileNewMenuItem() {  
2     if (fileNewMenuItem == null) {  
3         fileNewMenuItem = new LocalizedMenuItem(  
4             MenuItemNames.FILE_NEW, ActionCommands.NEW_NETWORK);  
5         fileNewMenuItem.setIcon(IconBind.NEW_ENABLED.icon());  
6         fileNewMenuItem.setAccelerator(  
7             KeyStroke.getKeyStroke(KeyEvent.VK_N, CTRL));  
8         fileNewMenuItem.addActionListener(listener);  
9     }  
10    return fileNewMenuItem;  
11 }  
12 // ... x43 métodos casi idénticos
```

Listado 5: Patrón repetitivo de construcción de ítems de menú

## 6.2. Problemas

1. **Repetición masiva**: 53 campos y 43 getters lazy con estructura idéntica.
2. **Mantenimiento**: añadir un ítem de menú requiere: declarar campo, escribir getter lazy, añadir al menú padre, registrar en `MenuItemNames`.
3. **Tamaño**: 1 529 líneas dificultan la navegación.

## 6.3. Estado: Completada

Se reemplazó la construcción imperativa por un `EnumMap<ActionCommands, JComponent>` con métodos factory:

```
1 private final Map<ActionCommands, JComponent> items =
2     new EnumMap<>(ActionCommands.class);
3
4 private void createItem(String name, ActionCommands action,
5                         IconBind icon, KeyStroke key) {
6     var item = new LocalizedMenuItem(
7         name, action.getCommandName(), icon, key);
8     item.addActionListener(listener);
9     items.put(action, item);
10 }
11
12 // 1 línea por ítem, en vez de 8:
13 createItem(FILE_NEW_MENUITEM, NEW_NETWORK, NEW_ENABLED, ctrl(
14     VK_N));
```

Listado 6: Solución aplicada: factory + EnumMap

El switch de 43 casos se reemplazó por `items.get(cmd)`. Resultado: 1 529 → 425 líneas (−72 %).

## 6.4. Ficheros afectados

- `menutoolbar/menu/MainMenu.java` — reescritura completa

# 7. R5 — Consolidación de ediciones de valor en CPT

## 7.1. Situación actual

Cuatro clases de edición (`PNEdit`) implementan la lógica de modificar una celda de una tabla de probabilidad condicional:

| Clase                             | Líneas       |
|-----------------------------------|--------------|
| TablePotentialValueEdit           | 327          |
| ICITablePotentialValueEdit        | 429          |
| AugmentedPotentialValueEdit       | 205          |
| LinkRestrictionPotentialValueEdit | 175          |
| <b>Total</b>                      | <b>1 136</b> |

Tabla 5: Ediciones de valor en CPT con lógica compartida.

Las cuatro comparten un patrón común: reciben un `Potential`, coordenadas (fila, columna), el valor nuevo, clonan la tabla anterior en `doEdit()`, actualizan la celda, y en `undo()` restauran la tabla clonada.

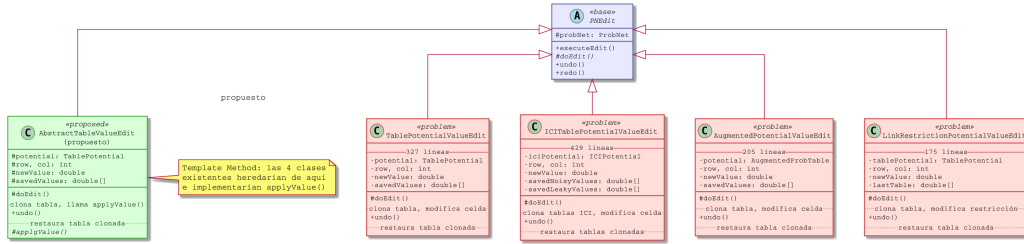


Figura 3: Jerarquía actual de las ediciones de valor en CPT. Las cuatro clases en rojo comparten la misma estructura interna. Diagrama desde `r5-edits-actual.puml`.

## 7.2. Problemas

1. **Duplicación**: el algoritmo de redistribución de probabilidades (ajustar valores para que sumen 1.0 según una lista de prioridad) se repetía en `TablePotentialValueEdit` (constructor) e `ICITablePotentialValueEdit` (`doEdit`). En esta última, el algoritmo aparecía duplicado internamente para parámetros *leaky* y *noisy*.
2. **Mantenimiento**: un cambio en la lógica de redondeo o redistribución debía replicarse en 3 puntos.

## 7.3. Solución aplicada

La propuesta original de crear una clase base `AbstractTableValueEdit` resultó inviable porque las cuatro clases usan dos patrones de herencia incompatibles: `TablePotentialValueEdit` y `AugmentedPotentialValueEdit` extienden `PotentialChangeEdit` (reemplazan el `potential` entero), mientras que `ICITablePotentialValueEdit` y `LinkRestrictionPotentialValueEdit` extienden `PNEdit` directamente. Además, `AugmentedPotentialValueEdit` no usa redistribución de probabilidades (trabaja con expresiones simbólicas) y `LinkRestrictionPotentialValueEdit` opera con valores binarios (0/1) sin normalización.

En su lugar, se extrajo el algoritmo de redistribución a un método utilitario estático en `PotentialsTablePanelOperations`:

```

1 public static void redistributeProbabilities(
2     double[] values, int editedPosition, double newValue,
3     List<Integer> priorityList, IntPredicate isEditable)

```

Listado 7: Método utilitario de redistribución de probabilidades

El predicado `isEditable` permite que `TablePotentialValueEdit` filtre posiciones no editables mientras que `ICITablePotentialValueEdit` pasa pos ->true.

## 7.4. Ficheros modificados

- `component/PotentialsTablePanelOperations.java` — nuevo método `redistributeProbabilities`
- `action/TablePotentialValueEdit.java` — eliminadas ~40 líneas de redistribución en el constructor
- `action/ICITablePotentialValueEdit.java` — eliminadas ~70 líneas (dos bloques duplicados leaky/noisy en `doEdit`)

## 8. R6 — Consolidación de `ValuesTable` y variantes

### 8.1. Situación actual

El paquete `component/` contiene una familia de tablas Swing para edición de CPT:

| Clase                                                        | Líneas       |
|--------------------------------------------------------------|--------------|
| <code>ValuesTable</code>                                     | 877          |
| <code>AugmentedValuesTable</code>                            | 267          |
| <code>ICIValuesTable</code>                                  | 142          |
| <code>LinkRestrictionValuesTable</code>                      | 111          |
| <code>ValuesTableModel</code>                                | 100          |
| <code>ValuesTableCellRenderer</code>                         | 233          |
| <code>ICIValuesTableCellRenderer</code>                      | 84           |
| <code>AugmentedValuesTableCellRenderer</code>                | 32           |
| <code>ValuesTableWithLinkRestrictionTableCellRenderer</code> | 33           |
| <code>ValuesTableOptimalPolicyTableCellRenderer</code>       | 40           |
| <b>Total</b>                                                 | <b>1 919</b> |

Tabla 6: Familia de tablas de CPT en `component/`.

`ValuesTable` (875 líneas) implementa `PNEditListener` y contiene lógica de renderizado, edición y undo/redo. Las variantes heredan de ella y sobrescriben partes.

## 8.2. Problemas

1. **Clase demasiado grande:** `ValuesTable` con 875 líneas mezcla modelo, vista y controlador dentro de un `JTable`.
2. **5 renderers:** la lógica de renderizado está dispersa en 5 clases separadas con duplicación parcial.
3. **Herencia frágil:** las variantes dependen de detalles internos de `ValuesTable`.

## 8.3. Estado: Completada

Tras analizar la jerarquía en detalle, las propuestas originales de consolidación arquitectónica (unificar renderers, extraer `PNEditListener`, reemplazar herencia por configuración) se descartaron por riesgo alto y beneficio limitado:

- Los 5 renderers implementan lógica de coloreado genuinamente diferente (ICI, política óptima, restricciones de enlace).
- Los callbacks `PNEditListener` están acoplados al modelo de datos de cada tabla (posiciones fila/columna de ediciones específicas).
- Las subclases usan correctamente `template method`: cada una sobrescribe `setValueAt()` y `afterEditExecutes()` con tipos de edición fundamentalmente distintos.

En su lugar, se realizó limpieza de código muerto y correcciones puntuales:

- **Clase eliminada:** `AugmentedValuesTableCellRenderer` (32 líneas) — nunca instanciada.
- **`AugmentedValuesTable`:** eliminados `printTable()`, `editCellAt()` (debug con `System.err.println`), `selectAll()` (copia privada inalcanzable) — -84 líneas.
- **`ValuesTable`:** eliminado `printTable()` — método debug nunca invocado — -21 líneas.
- **`ICIValuesTable`:** eliminado campo `lastCol` que sombreaba el campo `protected` del padre — -4 líneas.
- **`LinkRestrictionCellRenderer`:** eliminado parámetro `TablePotential` no utilizado. Actualizados 3 puntos de llamada en `LinkRestrictionPanel`.
- **`LinkRestrictionValuesTable`:** eliminada variable local `net` y `node1` no utilizadas — -9 líneas.

**Resultado:** 1 461 → 1 309 líneas (-152), 1 clase eliminada.

## 9. R7 — Simplificación de `VisualNetwork`

### 9.1. Situación actual

`VisualNetwork` (1 172 líneas) es la clase view-model que mantiene la representación visual de la red. Implementa `PNEditListener` y, en respuesta a cualquier edición, reconstruye toda la estructura visual:

```

1  @Override public void afterEditExecutes(PNEdit edit) {
2      constructVisualInfo();
3      if (getWorkingMode() != NetworkPanel.WorkingMode.INFERENCE)
4      {
5          visualDecisionNodeRefresh();
6      }
7      mainGUI.mainPanel.getEditionToolBar().getUndoButton()
8          .setEnabled(probNet.getPNESupport().getCanUndo());
9      mainGUI.mainPanel.getEditionToolBar().getRedoButton()
10         .setEnabled(probNet.getPNESupport().getCanRedo());
11 }
12 @Override public void afterUndoingEdit(PNEdit edit) {
13     refreshUI(); // mismo cuerpo que afterEditExecutes
14 }
15
16 @Override public void afterRedoingEdit(PNEdit edit) {
17     refreshUI(); // mismo cuerpo que afterEditExecutes
18 }

```

Listado 8: Métodos PNEditListener en VisualNetwork

## 9.2. Problemas

1. **Acoplamiento ascendente:** VisualNetwork (capa view-model) accede directamente a `mainGUI.mainPanel.getEditionToolBar()` (capa UI), violando la dirección de dependencias del MVC.
2. **Reconstrucción total:** `constructVisualInfo()` reconstruye todos los nodos y arcos visuales ante cualquier edición, sin distinguir qué cambió.
3. **Tamaño:** 1 177 líneas combinan mantenimiento de colecciones, geometría, hit-testing y notificación.

## 9.3. Estado: Completada

Se eliminó el campo `mainGUI` de `VisualNetwork` y las 4 líneas que manipulaban directamente los botones undo/redo de `EditionToolBar`. La actualización de estos botones ya la realiza `MainPanelMenuAssistant` mediante su método `updateUndoRedo()`, que invoca `setOptionEnabled(ActionCommands.UNDO/REDO, ...)`. Este método, heredado de `MenuAssistant`, itera sobre todas las implementaciones de `MenuToolBarBasic` registradas — incluyendo tanto `MainMenu` como las clases de toolbar — por lo que los botones de toolbar ya se actualizaban por esta vía.

El constructor de `VisualNetwork` pasó de `VisualNetwork(ProbNet, MainGUI)` a `VisualNetwork(ProbNet)`. Se actualizó el único punto de llamada en `NetworkPanel`.

No fue necesario registrar `EditionToolBar` como `PNEditListener` independiente, ya que el mecanismo existente en `MainPanelMenuAssistant` cubre tanto menús como toolbars.

- `graphic/VisualNetwork.java` — 1 172 → 1 161 líneas (−11)
- `window/edition/NetworkPanel.java` — eliminado argumento `mainGUI` en constructor de `VisualNetwork`

## 10. R8 — Consolidación de editores temporales

### 10.1. Situación actual

Tres clases de edición gestionan intervalos temporales con estructura similar:

| Clase                                    | Líneas     |
|------------------------------------------|------------|
| <code>NodePartitionedIntervalEdit</code> | 165        |
| <code>RevelationIntervalEdit</code>      | 152        |
| <code>PartitionedIntervalEdit</code>     | 47         |
| <b>Total</b>                             | <b>364</b> |

Tabla 7: Ediciones temporales con estructura compartida.

Las tres manipulan rangos temporales (`timeSlice`) sobre nodos y comparten la lógica de validación de intervalos y undo.

### 10.2. Problemas

1. **Duplicación parcial**: la validación de intervalos y la lógica de undo se repite con variaciones menores.
2. **Baja prioridad**: el volumen total es pequeño (364 líneas) y la duplicación es parcial, no total.

### 10.3. Decisión: descartada

Tras analizar el código, las tres clases no comparten suficiente estructura para justificar una abstracción común: `PartitionedIntervalEdit` (47 líneas) es un simple swap de intervalos; `NodePartitionedIntervalEdit` modifica propiedades in-place; `RevelationIntervalEdit` manipula una lista en un link con 4 acciones distintas (`ADD`, `REMOVE`, `MODIFY_VALUE`, `MODIFY_DELIMITER`). Los modelos de datos y las estrategias de backup son incompatibles. La duplicación parcial no justifica el coste de una abstracción para 364 líneas.

## 11. Orden de ejecución recomendado

Las refactorizaciones se han ordenado considerando: (1) impacto en mantenibilidad, (2) riesgo de regresión, (3) dependencias entre ellas.

1. **R2** — **Completada**. Las tres subclases fueron eliminadas y `NodeEditionMode` es ahora instanciable directamente.
2. **R3** (casts `TablePotential`) — **Completada**. Cambiada la firma de `Link.getRestrictionsPotent` para devolver `TablePotential`; eliminados 15 de 16 casts con pattern matching `instanceof`.
3. **R5** (ediciones de valor CPT) — **Completada**. Extraído algoritmo de redistribución de probabilidades a `PotentialsTablePanelOperations.redistributeProbabilities`; eliminadas ~110 líneas duplicadas.
4. **R8** (editores temporales) — **Descartada**. Las 3 clases no comparten suficiente estructura para justificar una abstracción (ver §10).
5. **R4** (`MainMenu`) — **Completada**. 53 campos y 46 getters lazy reemplazados por `EnumMap<ActionCommands, JComponent>` con métodos `factory`; switch de 43 casos reemplazado por `items.get(cmd)`; 1529 → 425 líneas (−72 %).
6. **R1** (`MainPanelListenerAssistant`) — **Completada**. Descompuesto en fachada (401 líneas) + 3 handlers package-private: `NetworkFileHandler` (467), `InferenceHandler` (238), `EditAndViewHandler` (121).
7. **R7** (`VisualNetwork`) — **Completada**. Eliminado campo `mainGUI` y acceso directo a toolbar; la actualización undo/redo ya la gestiona `MainPanelMenuAssistant`.
8. **R6** (`ValuesTable`) — **Completada**. Limpieza de código muerto: 1 clase eliminada, métodos debug, campo sombreado, parámetro no utilizado; −152 líneas. Consolidación arquitectónica descartada por alto riesgo.

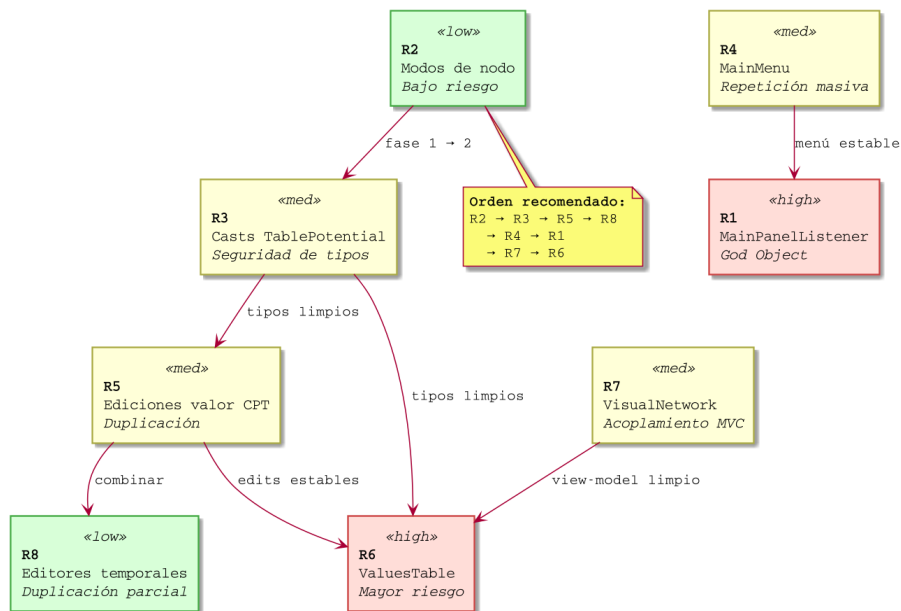


Figura 4: Grafo de dependencias entre refactorizaciones. Las flechas indican “debería hacerse antes de”. Diagrama desde `orden-ejecucion.puml`.



## 12. Resumen cuantitativo

| Refact.      | Clases    | Líneas        | Reducción est. | Beneficio principal |
|--------------|-----------|---------------|----------------|---------------------|
| R1           | 4         | 1 534         | −307           | Completada          |
| R2           | 5         | 527           | −44            | Completada          |
| R3           | 10        | 2 500         | −200           | Completada          |
| R4           | 1         | 1 529         | −1 104         | Completada          |
| R5           | 3         | 1 136         | −110           | Completada          |
| R6           | 7         | 1 461         | −152           | Completada          |
| R7           | 2         | 1 172         | −11            | Completada          |
| R8           | 3         | 364           | 0              | Descartada          |
| <b>Total</b> | <b>34</b> | <b>10 223</b> | <b>−1 517</b>  |                     |

Tabla 8: Resumen cuantitativo de las refactorizaciones propuestas. La columna “Reducción est.” indica la reducción estimada en líneas de código (neto, sin contar nuevas clases base).